LAB-14.5

Web Design Application – AI Assisted HTML/CSS/JS Generation

2303A51221

B.TECH CSE

BATCH – 04

TASK 1– Image Gallery Website

Create an image gallery web page.

Requirements:

- **Display images in a grid layout.**
- **Use CSS Grid.**
- **Add hover zoom effect.**
- **Make it mobile responsive.**

PROMPT

Generate responsive HTML and CSS code for an image gallery using CSS Grid with hover zoom effects and mobile responsiveness.

CODE

```
<!DOCTYPE html>
<html>
<head>
<title>Image Gallery</title>
<style>
body{
font-family: Arial;
text-align:center;
}
.gallery{
display:grid;
grid-template-columns:repeat(auto-fit,minmax(200px,1fr));
gap:15px;
padding:20px;
}
.gallery img{
width:100%;
```

```
border-radius:10px;
transition:0.3s;
}
.gallery img:hover{
transform:scale(1.1);
}
</style>
</head>
<body>
<h2>Image Gallery</h2>
<div class="gallery">
<img src="https://picsum.photos/300/200?1">
<img src="https://picsum.photos/300/200?2">
<img src="https://picsum.photos/300/200?3">
<img src="https://picsum.photos/300/200?4">
</div>
</body>
</html>
```

```html
<!DOCTYPE html>
<html>
<head>
<title>Image Gallery</title>

<style>

body{
font-family: Arial;
text-align:center;
}

.gallery{
display:grid;
grid-template-columns:repeat(auto-fit,minmax(200px,1fr));
gap:15px;
padding:20px;
}

.gallery img{
width:100%;
border-radius:10px;
transition:0.3s;
}

.gallery img:hover{
transform:scale(1.1);
}

</style>

</head>

<body>

<h2>Image Gallery</h2>

<div class="gallery">

<img src="https://picsum.photos/300/200?1">
<img src="https://picsum.photos/300/200?2">
<img src="https://picsum.photos/300/200?3">
<img src="https://picsum.photos/300/200?4">

</div>

</body>
</html>
```

RESULT

Images are displayed in a grid format.

OUTPUT

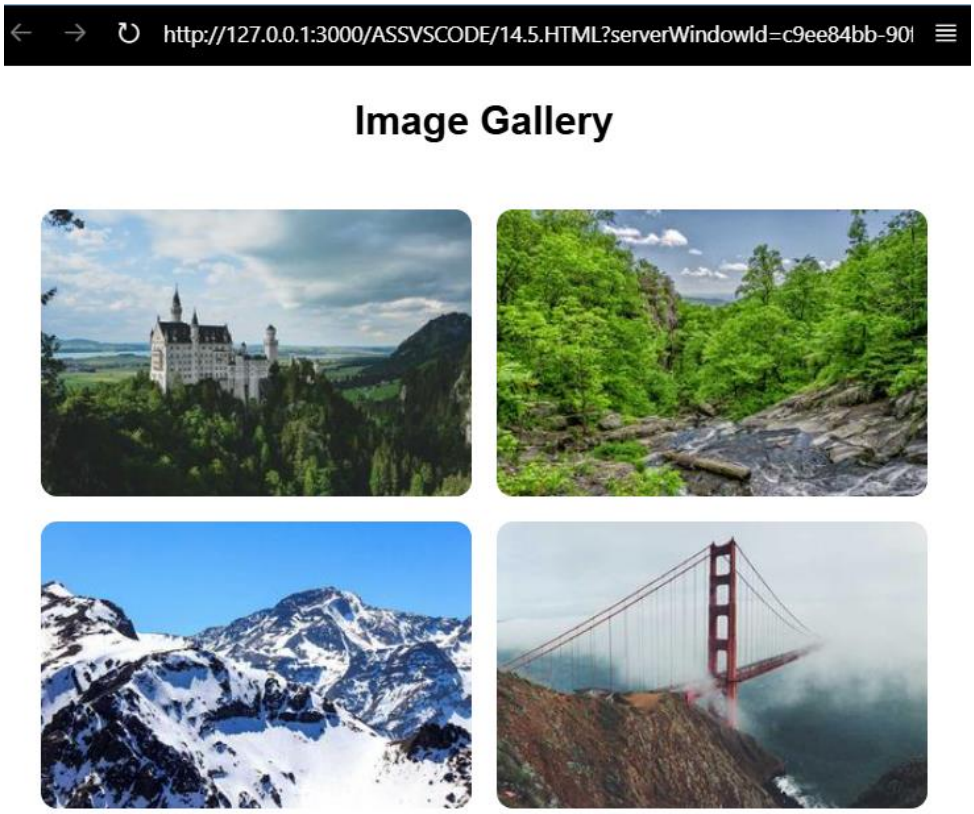


Image gallery appears with hover zoom effect.

EXPLANATION

CSS Grid arranges the images automatically in rows and columns. The hover transform enlarges images slightly to create an interactive UI.

TASK 2 – Profile Card Generator

QUESTION

Create a profile card displaying:

- User image
- Name
- Role
- Social links

Using Flexbox with hover animation.

PROMPT

Generate a modern profile card using HTML and CSS Flexbox with hover effects and shadows.

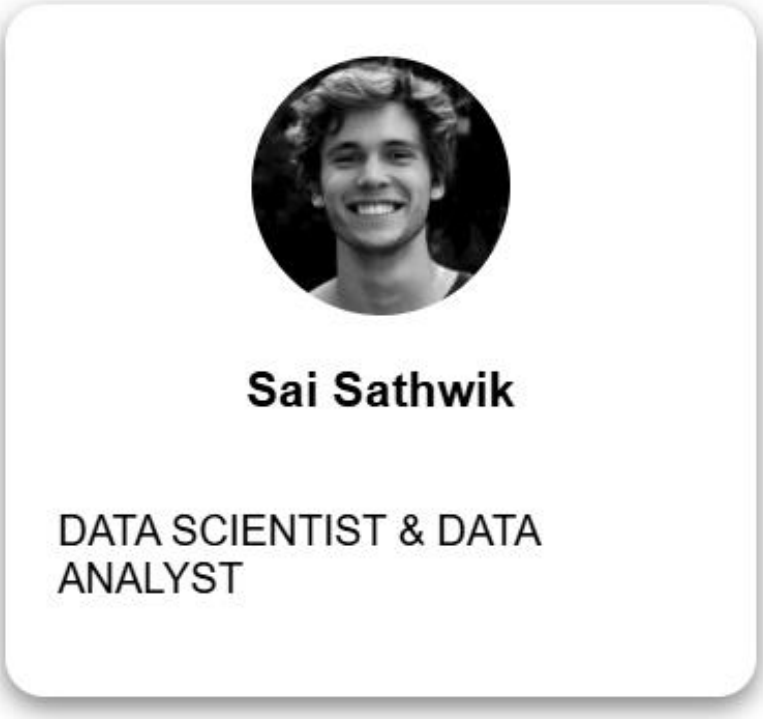
CODE

```html
<!DOCTYPE html>
<html>
<head>
<style>
body{
font-family:Arial;
display:flex;
justify-content:center;
margin-top:50px;
}
.card{
width:250px;
padding:20px;
border-radius:15px;
box-shadow:0 4px 10px gray;
display:flex;
flex-direction:column;
align-items:center;
transition:0.3s;
}
.card:hover{
transform:translateY(-8px);
}
img{
width:100px;
border-radius:50%;
}
</style>
</head>
<body>
<div class="card">
<img src="https://i.pravatar.cc/150">
<h3>Sai Sathwik</h3>
<p>DATA SCIENTIST & DATA ANALYST</p>
</div>
</body>
</html>
```

RESULT

A modern profile card appears.

OUTPUT



Card lifts upward when hovering.

EXPLANATION

Flexbox aligns elements vertically while hover animation improves visual interaction.

TASK 3 – Dynamic Shopping Cart System

QUESTION

Develop a shopping cart system that:

- **Adds products**
- **Removes products**
- **Updates quantity**
- **Displays real-time total price**
- **Stores data in local Storage.**

PROMPT

Generate JavaScript code for a shopping cart with add/remove functionality and total price calculation using local Storage.

CODE

```
<!DOCTYPE html>
<html>
<head>
<title>SHOPPING BASKET</title>
</head>
<body>
<h2>Shopping Cart</h2>
<button onclick="addItem('Product',100)">Add Product</button>
<p>Total Price: ₹<span id="total">0</span></p>
<script>
let cart=[];
function addItem(name,price){
cart.push({name,price});
updateTotal();
}
function updateTotal(){
let total=cart.reduce((sum,item)=>sum+item.price,0);
document.getElementById("total").innerText=total;
localStorage.setItem("cart",JSON.stringify(cart));
}
</script>
</body>
</html>
```

RESULT

Products can be added and total price updates automatically.

OUTPUT

SHOPPING BASKET

Add Product

Total Price: ₹0

Total price increases when items are added.

EXPLANATION

JavaScript objects store product data and local Storage preserves cart information.

TASK 4 – Role Based Access Control

QUESTION

Create roles:

- **Admin**
- **Editor**
- **User**

Different UI should appear for each role.

PROMPT

Generate JavaScript logic for role-based access control with conditional UI rendering.

CODE

```
<!DOCTYPE html>
<html>
<body>
<h2>Role Based Dashboard</h2>
<button onclick="showDashboard('admin')">Admin</button>
<button onclick="showDashboard('editor')">Editor</button>
<button onclick="showDashboard('user')">User</button>
<p id="output"></p>
<script>
function showDashboard(role){
if(role=="admin"){
document.getElementById("output").innerText="Admin Dashboard";
}
else if(role=="editor"){
document.getElementById("output").innerText="Editor Dashboard";
}
else{
document.getElementById("output").innerText="User Dashboard";
}
}
</script>
</body>
</html>
```

RESULT

Different dashboards appear depending on role.

OUTPUT

Role Based Dashboard

Admin Editor User

User Dashboard

Role Based Dashboard

Admin Editor User

Editor Dashboard

Role Based Dashboard

Admin Editor User

Admin Dashboard

Admin / Editor / User dashboard message displays.

Explanation

Conditional statements check the role and render the appropriate interface.

TASK 5 – Multi Language Website

QUESTION

Create a website that:

- **Switches languages**
- **Stores language preference.**

PROMPT

Generate JavaScript code for dynamic language switching with local Storage.

CODE

```html
<!DOCTYPE html>
<html>
<body>
<h2 id="title">Welcome</h2>
<button onclick="setLanguage('en')">English</button>
<button onclick="setLanguage('es')">Spanish</button>
<script>
const translations={
en:{title:"Welcome"},
es:{title:"Bienvenido"}
}
function setLanguage(lang){
document.getElementById("title").innerText=translations[lang].title;
localStorage.setItem("lang",lang)
}
</script>
</body>
</html>
```

RESULT

Language changes dynamically.

OUTPUT

Welcome

English Spanish

Bienvenido

English Spanish

Title changes between English and Spanish.

EXPLANATION

Translation objects store text for each language and JavaScript updates the UI.

Task 6 – Online Examination System

QUESTION

Create an exam interface with:

- **Timer**
- **Questions**
- **Submission**
- **Prevent multiple submissions.**

PROMPT

Generate JavaScript code for an exam timer with automatic submission.

```html
<!DOCTYPE html>
<html>

<body>

<h2>Online Exam</h2>

<p>Time Left: <span id="timer">30</span></p>

<button onclick="submitExam()">Submit</button>

<script>

let time=30;

let timer=setInterval(function(){

document.getElementById("timer").innerText=time;

time--;
if(time<0){
clearInterval(timer);
submitExam();
}
},1000);
function submitExam(){
alert("Exam Submitted");
}
</script>
</body>
</html>
```

RESULT

Timer counts down and exam is submitted automatically.

OUTPUT

> # Online Exam
>
> Time Left: 12
>
> Submit

Alert message shows "Exam Submitted".

EXPLANATION

A countdown timer controls exam duration and ensures automatic submission.